

AquaSave: Documentación de análisis, diseño y técnica

Centro Educativo Técnico Laboral Kinal · Taller · Profesor: Alejandro Max · 5to Perito Informática

Autor: Michael Yohardi Aquino Arqueta · Carné 2025436 · Versión 1.0 (borrador)

1. Análisis de la problemática y alcance

1.1 Problemática mundial

Las fugas de agua potable generan pérdidas enormes de un recurso escaso. Muchas permanecen sin reportar o se atienden tarde, lo que aumenta el desperdicio y los costos de reparación. El proyecto se relaciona con el **Objetivo de Desarrollo Sostenible 6 (Agua limpia y saneamiento)** y con el **ODS 11 (Ciudades y comunidades sostenibles)**.

1.2 Población beneficiaria y usuarios

- **Ciudadanos:** habitantes que detectan fugas y quieren reportarlas y seguir su atención.
- **Administradores (autoridades o entidad responsable del agua):** reciben los reportes, priorizan, asignan cuadrillas y miden resultados.

1.3 Objetivos

- **General:** desarrollar y desplegar una plataforma web que permita reportar y gestionar fugas de agua potable.
- **Específicos:** registrar reportes con evidencia, priorizar incidencias por gravedad, mejorar los tiempos de respuesta con seguimiento de estados y promover la participación ciudadana.

1.4 Alcance

Incluido	Excluido
Registro e inicio de sesión con roles	Aplicación móvil nativa
Reportes con ubicación, descripción, fotos y gravedad	Pagos o facturación del servicio de agua
Asignación de cuadrillas y seguimiento de estados	Notificaciones por correo o SMS
Incluido	Excluido

Panel de indicadores y administración de catálogos	Mapa interactivo integrado (solo enlace a mapa externo)
--	---

1.5 Requisitos funcionales del sistema

Código	Requisito
RF-01	Registro e inicio de sesión de usuarios, con cierre de sesión.
RF-02	Contraseñas almacenadas con hash seguro (bcrypt).
RF-03	Autenticación con JWT.
RF-04	Dos roles: ciudadano y administrador, con permisos distintos.
RF-05	Rutas del frontend y endpoints protegidos según autenticación y rol.
RF-06	Perfil de usuario consultable y editable.
RF-07	Entidades de negocio: zonas, tipos de fuga, cuadrillas y reportes (más evidencias y asignaciones).
RF-08	CRUD completo de zonas, tipos de fuga, cuadrillas y reportes.
RF-09	Proceso transaccional: reporte, asignación de cuadrilla, reparación y cierre, con historial.
RF-10	Búsqueda, filtros, orden y paginación en el listado de reportes.
RF-11	Dashboard con indicadores calculados con datos reales.
RF-12	Validaciones en frontend y backend con mensajes comprensibles.
RF-13	Estados del reporte: pendiente, en revisión, asignado, en reparación, resuelto y rechazado.
RF-14	Confirmación antes de eliminar.
Código	Requisito
RF-15	Estados visibles de carga, éxito, lista vacía y error.

1.6 Requisitos no funcionales

Categoría	Criterio aplicado
Seguridad	Hash de contraseñas, JWT, autorización en el servidor, validación con Zod, consultas parametrizadas, secretos en <code>.env</code> , límite de intentos en login, cabeceras con Helmet.
Usabilidad	Flujos cortos, mensajes en español, confirmación de acciones irreversibles.
Rendimiento	Listados paginados, índices en columnas de búsqueda.
Compatibilidad	Chrome, Edge y Firefox actuales.
Accesibilidad	Contraste mínimo 4.5:1, etiquetas en formularios, foco visible, navegación por teclado.
Privacidad	Solo datos ficticios en pruebas y demostración.

1.7 Casos de uso principales

Caso de uso	Actor	Descripción
Registrarse e iniciar sesión	Ciudadano, administrador	Crear cuenta y acceder con correo y contraseña.
Reportar una fuga	Ciudadano	Llenar el formulario con zona, tipo, descripción, dirección, gravedad y fotos.
Dar seguimiento	Ciudadano	Consultar el estado y el historial de sus reportes.
Asignar cuadrilla	Administrador	Elegir una cuadrilla disponible para un reporte pendiente o en revisión.
Actualizar estado	Administrador	Avanzar el reporte hasta resuelto o rechazarlo.
Caso de uso	Actor	Descripción

Administrar catálogos y usuarios	Administrador	CRUD de zonas, tipos y cuadrillas; activar o desactivar usuarios.
Consultar indicadores	Administrador, ciudadano	Ver totales, urgentes, resueltos y tiempo promedio.

2. Planificación y organización

Fase	Actividades	Estado
1. Diseño	Análisis, modelo de datos, paleta visual	Completada
2. Desarrollo	Backend (API, seguridad, proceso principal) y frontend Angular	En curso
3. Integración y pruebas	Pruebas en Postman, Vitest y navegador	En curso
4. Implementación	Despliegue y documentación final	Pendiente

Repositorio: monorepo con las carpetas `backend`, `frontend` y `docs`. Los commits se hacen de forma progresiva por módulo (autenticación, catálogos, reportes, proceso de asignación, frontend).

Riesgos identificados: baja adopción de usuarios, problemas de conectividad y retrasos en el desarrollo.

3. Arquitectura

```

Navegador (Angular 21, TypeScript)
  |   HTTPS, JSON (REST) + JWT en el encabezado Authorization
v
API REST (Node.js + Express + TypeScript)
  rutas -> validación (Zod) -> servicios -> acceso a datos (mysql2)
  |   consultas parametrizadas
  v
Base de datos MySQL           Carpeta /uploads (fotos de los reportes)

```

Flujo de datos: el usuario interactúa con Angular, el interceptor HTTP agrega el token a cada petición, la API valida el token, el rol y los datos, ejecuta la consulta en MySQL y responde en JSON. Angular muestra el resultado con estados de carga, error, éxito o lista vacía.

4. Base de datos (MySQL)

Justificación del motor: se eligió MySQL por ser ampliamente soportado por proveedores de alojamiento, ligero y suficiente para un modelo relacional como este. Ofrece integridad referencial, transacciones (InnoDB) e índices.

4.1 Modelo entidad relación

Relación	Cardinalidad
roles y usuarios	1 a N
usuarios y reportes	1 a N
zonas y reportes	1 a N
tipos_fuga y reportes	1 a N
zonas y cuadrillas	1 a N (opcional)
reportes y evidencias	1 a N
reportes y historial_estados	1 a N
usuarios y historial_estados	1 a N
reportes y cuadrillas	N a M, resuelta con la tabla intermedia asignaciones
usuarios y asignaciones (asignado_por)	1 a N

Normalización: el modelo cumple la tercera forma normal. Los catálogos (roles, zonas, tipos de fuga, cuadrillas) están en tablas propias y los reportes los referencian por clave foránea. Excepción justificada: `historial_estados` guarda el nombre del estado como texto porque es una bitácora histórica que no debe cambiar si el modelo evoluciona.

Índices: `reportes` tiene índices en `estado`, `gravedad`, `zona_id` y `creado_en` para acelerar filtros y orden. Las columnas `email` (usuarios) y `nombre` (catálogos) son únicas.

4.2 Diccionario de datos roles

Campo	Tipo	Nulo	Restricciones	Descripción
id	TINYINT UNSIGNED	No	PK, auto incremento	Identificador del rol

nombre	VARCHAR(30)	No	Único	ciudadano o administrador
--------	-------------	----	-------	---------------------------

usuarios

Campo	Tipo	Nulo	Restricciones	Descripción
id	INT UNSIGNED	No	PK, auto incremento	Identificador
rol_id	TINYINT UNSIGNED	No	FK a roles	Rol del usuario
nombre	VARCHAR(100)	No		Nombre completo
email	VARCHAR(120)	No	Único	Correo para iniciar sesión
telefono	VARCHAR(20)	Sí		Teléfono opcional
password_hash	VARCHAR(255)	No		Contraseña cifrada con bcrypt
activo	BOOLEAN	No	Por defecto TRUE	Si puede iniciar sesión
creado_en	TIMESTAMP	No	Por defecto la fecha actual	Fecha de registro

zonas

Campo	Tipo	Nulo	Restricciones	Descripción
id	INT UNSIGNED	No	PK, auto incremento	Identificador
nombre	VARCHAR(100)	No	Único	Nombre de la zona
municipio	VARCHAR(100)	No		Municipio al que pertenece
activa	BOOLEAN	No	Por defecto TRUE	Si está en uso

tipos_fuga

Campo	Tipo	Nulo	Restricciones	Descripción
id	INT UNSIGNED	No	PK, auto incremento	Identificador
nombre	VARCHAR(80)	No	Único	Nombre del tipo de fuga
descripcion	VARCHAR(255)	Sí		Detalle del tipo

cuadrillas

Campo	Tipo	Nulo	Restricciones	Descripción
id	INT UNSIGNED	No	PK, auto incremento	Identificador
nombre	VARCHAR(100)	No	Único	Nombre de la cuadrilla
telefono	VARCHAR(20)	Sí		Contacto
zona_id	INT UNSIGNED	Sí	FK a zonas, ON DELETE SET NULL	Zona de trabajo habitual
disponible	BOOLEAN	No	Por defecto TRUE	Si puede recibir asignaciones

reportes

Campo	Tipo	Nulo	Restricciones	Descripción
id	INT UNSIGNED	No	PK, auto incremento	Identificador
usuario_id	INT UNSIGNED	No	FK a usuarios	Quien reporta
zona_id	INT UNSIGNED	No	FK a zonas	Zona de la fuga
tipo_fuga_id	INT UNSIGNED	No	FK a tipos_fuga	Tipo de fuga
titulo	VARCHAR(150)	No		Título corto
descripcion	TEXT	No		Descripción detallada
direccion	VARCHAR(255)	No		Dirección o referencia
latitud	DECIMAL(9,6)	Sí		Coordenada opcional
Campo	Tipo	Nulo	Restricciones	Descripción
longitud	DECIMAL(9,6)	Sí		Coordenada opcional

gravedad	ENUM	No	baja, media, alta, critica; por defecto media	Nivel de gravedad
estado	ENUM	No	pendiente, en_revision, asignado, en_reparacion, resuelto, rechazado; por defecto pendiente	Estado del proceso
creado_en	TIMESTAMP	No	Por defecto la fecha actual	Fecha de creación
actualizado_en	TIMESTAMP	No	Se actualiza automáticamente	Última modificación

evidencias

Campo	Tipo	Nulo	Restricciones	Descripción
id	INT UNSIGNED	No	PK, auto incremento	Identificador
reporte_id	INT UNSIGNED	No	FK a reportes, ON DELETE CASCADE	Reporte al que pertenece
ruta_archivo	VARCHAR(255)	No		Nombre del archivo guardado
subido_en	TIMESTAMP	No	Por defecto la fecha actual	Fecha de carga

asignaciones

Campo	Tipo	Nulo	Restricciones	Descripción
id	INT UNSIGNED	No	PK, auto incremento	Identificador
reporte_id	INT UNSIGNED	No	FK a reportes	Reporte atendido
cuadrilla_id	INT UNSIGNED	No	FK a cuadrillas	Cuadrilla asignada
asignado_por	INT UNSIGNED	No	FK a usuarios	Administrador que asignó
notas	VARCHAR(255)	Sí		Indicaciones
Campo	Tipo	Nulo	Restricciones	Descripción

fecha_asignacion	TIMESTAMP	No	Por defecto la fecha actual	Inicio de la asignación
fecha_cierre	TIMESTAMP	Sí		Se llena al resolver

historial_estados

Campo	Tipo	Nulo	Restricciones	Descripción
id	INT UNSIGNED	No	PK, auto incremento	Identificador
reporte_id	INT UNSIGNED	No	FK a reportes, ON DELETE CASCADE	Reporte
usuario_id	INT UNSIGNED	No	FK a usuarios	Quien hizo el cambio
estado_anterior	VARCHAR(20)	Sí		Estado previo
estado_nuevo	VARCHAR(20)	No		Estado nuevo
comentario	VARCHAR(255)	Sí		Observación del cambio
fecha	TIMESTAMP	No	Por defecto la fecha actual	Momento del cambio

La estructura se crea desde cero con `npm run db:init (script backend/db/schema.sql )` y los datos de demostración con `npm run seed .`

5. Backend

Stack: Node.js, TypeScript, Express 5, mysql2, Zod, jsonwebtoken, bcrypt, Helmet, CORS, express-rate-limit y Multer.

Organización en `backend/src` : `config` (conexión a la base), `middlewares` (autenticación, roles, subida de archivos y manejo de errores), `modules` (auth, catálogos, reportes, dashboard, usuarios) y `utils` (errores, transacciones y generador de CRUD).

5.1 Endpoints

Todos requieren token (`Authorization: Bearer ...`) salvo los marcados como públicos.

Método y ruta	Acceso	Descripción
GET /health	Público	Estado de la API
POST /api/auth/registro	Público	Crear cuenta de ciudadano
POST /api/auth/login	Público	Iniciar sesión, devuelve el token
GET /api/auth/perfil	Autenticado	Consultar el perfil propio
PUT /api/auth/perfil	Autenticado	Actualizar nombre y teléfono
GET /api/zonas, /api/tipos-fuga, /api/cuadrillas	Autenticado	Listar con búsqueda (q) y paginación (page , limit)
GET /api/{recurso}/:id	Autenticado	Obtener por id
POST, PUT, DELETE /api/{recurso}	Administrador	Crear, actualizar y eliminar
GET /api/reportes	Autenticado	Listar (el ciudadano ve solo los suyos). Filtros: q , estado , gravedad , zona_id , orden , page , limit
GET /api/reportes/:id	Autenticado	Detalle con evidencias, historial y asignaciones
POST /api/reportes	Autenticado	Crear reporte (multipart, hasta 3 fotos)
PUT /api/reportes/:id	Autenticado	Editar (el ciudadano solo si está pendiente)
DELETE /api/reportes/:id	Autenticado	Eliminar (el ciudadano solo si está pendiente)
POST /api/reportes/:id/asignar	Administrador	Asignar cuadrilla (transacción)
PATCH /api/reportes/:id/estado	Administrador	Cambiar estado según las transiciones permitidas

GET /api/dashboard	Autenticado	Indicadores (globales para el administrador, propios para el ciudadano)
GET /api/usuarios	Administrador	Listar usuarios con búsqueda y paginación
PATCH /api/usuarios/:id	Administrador	Cambiar rol o activar o desactivar

El cierre de sesión se hace en el cliente: Angular borra el token guardado. La documentación de la API está en la colección de Postman

`docs/AquaSave.postman_collection.json` .

5.2 Proceso principal y estados

Transiciones permitidas: pendiente a en revisión o rechazado; en revisión a rechazado (o a asignado mediante la asignación de cuadrilla); asignado a en reparación; en reparación a resuelto. Los estados resuelto y rechazado son finales.

La asignación y el cambio de estado se ejecutan dentro de una transacción con bloqueo de filas (`FOR UPDATE`): registran la asignación, actualizan la disponibilidad de la cuadrilla, el estado del reporte y el historial. Si una parte falla, todo se revierte.

6. Frontend (Angular)

Stack: Angular 21 con componentes independientes, señales, formularios reactivos, enrutamiento con carga diferida, interceptor y guards funcionales. Gestor de paquetes: `pnpm`.

Estructura en `frontend/src/app` : `core` (modelos, servicios, interceptor, guards y utilidades), `layout` (menú y barra superior), `features` (auth, reportes, dashboard, admin y perfil) y `shared` (etiquetas y diálogo de confirmación).

Ruta	Acceso
/login y /registro	Solo sin sesión
/dashboard, /reportes, /reportes/nuevo, /reportes/:id, /reportes/:id/editar	Con sesión
/perfil	Con sesión
/admin/*	Solo administrador

La seguridad real está en el servidor: los guards del frontend solo mejoran la experiencia.

7. Identidad visual

Temática de agua, con colores azul y verde azulado. Los contrastes se calcularon con la fórmula WCAG.

Elemento	Nombre	HEX	Uso	Contraste
Primario	Azul agua	#0B5FA5	Botones principales, enlaces y barra superior	6.57:1 con blanco
Primario (hover)	Azul profundo	#084B85	Estado hover de botones	8.92:1 con blanco
Secundario	Verde azulado	#0F766E	Acciones secundarias, filtros y categorías	5.47:1 con blanco
Acento	Ámbar	#F59E0B	Resaltar información importante (uso limitado)	6.86:1 con texto #3B2300
Fondo	Niébla	#F4F8FB	Fondo de la aplicación	13.86:1 con el texto
Superficie	Blanco	#FFFFFF	Tarjetas, formularios y tablas	14.8:1 con el texto
Borde	Gris azulado	#D5E0E8	Divisores y bordes	Decorativo
Borde de campos	Gris acero	#6B8294	Bordes de campos de formulario	4.0:1 con blanco
Texto	Azul noche	#0F2A3D	Texto principal	14.8:1 sobre blanco
Texto suave	Gris azulado	#4A6275	Texto secundario	6.37:1 sobre blanco
Menú lateral	Azul marino	#0A2F4A	Fondo del menú	11.97:1 con #E6F0F7

Éxito	Verde	#14532D sobre #DCFCE7	Mensajes de éxito y estado resuelto	8.3:1
Advertencia	Ámbar	#713F12 sobre #FEF3C7	Avisos y estado pendiente	7.79:1
Error	Rojo	#7F1D1D sobre #FEE2E2	Errores y estado rechazado	8.2:1
Información	Azul	#1E3A8A sobre #DBEAFE	Mensajes informativos	8.49:1

Tipografía: Segoe UI, Roboto, Helvetica, Arial, sans-serif. Título principal 1.75 rem, subtítulos 1.4 y 1.15 rem, texto base 1 rem y texto secundario 0.875 rem.

Estados sin depender solo del color: los mensajes llevan un prefijo de texto (Error, Éxito, Aviso, Info), los errores de campo llevan el símbolo ✖ y cada estado de un reporte muestra su nombre escrito.

8. Seguridad

- Contraseñas con bcrypt, nunca en texto plano.
- JWT firmado con un secreto guardado en `.env` (no se sube a Git; se incluye `.env.example`). Rol verificado en el servidor en cada endpoint protegido.
- Validación de datos de entrada con Zod y consultas parametrizadas contra inyección SQL.
- CORS limitado al origen del frontend, tamaño máximo de petición de 1 MB y límite de intentos en el login.
- Subida de imágenes limitada a JPG, PNG y WEBP de hasta 5 MB, con nombre aleatorio.
- Manejo central de errores sin exponer detalles internos.
- Un ciudadano no puede ver reportes ajenos: el servidor responde 404.

9. Decisiones técnicas y limitaciones

- Se usó SQL parametrizado con `mysql2` en lugar de un ORM, por simplicidad y control de las transacciones.
- Las fotos se guardan en el disco del servidor; en un servicio de alojamiento con disco temporal pueden perderse al reiniciar. Para producción real conviene un almacenamiento externo.
- Un usuario desactivado conserva su token hasta que expire (2 horas); el login sí queda bloqueado.
- No hay recuperación de contraseña ni notificaciones por correo.
- Si una foto se sube y la validación del reporte falla, el archivo puede quedar huérfano en el servidor.

10. Instalación y ejecución

Requisitos: Node.js 20 o superior, MySQL 8, npm y pnpm.

```
git clone
https://github.com/Michaelby2008
/AquaSafe.git

cd AquaSafe

# Backend
cd backend
npm install
copy .env.example .env      # completar con los valores reales
npm run db:init              # crea la base de datos y las tablas
npm run seed                 # datos de demostración npm
run dev                      # API en http://localhost:3000

# Frontend (otra terminal)
cd frontend pnpm install pnpm start
# http://localhost:4200
```

Variables del backend: PORT , DB_HOST , DB_PORT , DB_USER , DB_PASSWORD , DB_NAME , JWT_SECRET , JWT_EXPIRES_IN , CORS_ORIGIN . El frontend toma la URL de la API de src/environments .

Pruebas del backend: npm test . Despliegue: <http://localhost:4200>, <http://localhost:3000/>.

11. Credenciales de demostración

Rol	Correo	Contraseña
Administrador	admin@aquasave.test	Demo1234!
Ciudadano	ciudadano@aquasave.test	Demo1234!

Son cuentas ficticias creadas por el script de datos iniciales. Las credenciales reales de administración del servidor y de la base de datos no se publican.

12. Pruebas

El plan de pruebas está en docs/plan-de-pruebas.md : 30 casos manuales en Postman con evidencias, 8 pruebas automatizadas del backend con Vitest y el registro de incidencias.